

Scalable Parallel-in-Time Integration for Equations of Motion: Particle Production in Analog Cosmology

Nathaniel Chapman¹

¹Department of Computer Science, Central Washington University

April 25, 2025

Abstract

Simulating time-dependent physics has traditionally been constrained to using sequential algorithms, thus not benefiting from advances in parallel computing. Parallel-in-time integration attempts to address this limitation with methods such as the Parareal algorithm. As the performance of the Parareal algorithm scales with the number of processors, it is well-suited to use the massively-parallel nature of graphics processing units. Additional performance gains are seen when the physics is wave-like, as using a spectral method allows for each node in a distributed system to evaluate the Parareal algorithm. Particle production in different cosmologies is used to highlight the performance gains from these methods.

*This work is dedicated to
my friends for sharing laughs and rants,
Mr. Chris Lacey for making physics phun,
Dr. Brandon Peden for showing me how to be a physicist,
and Dr. Andy Piacsek for making sure I finish this damn thing.*

Contents

1. Introduction	4
2. Background	5
2.1. Equations of Motion	6
2.1.1. Initial Value Differential Equations	6
2.1.2. Traditional Numerical Methods	6
2.2. High-Performance Computing	7
2.2.1. Multi-threading & GPU Computing	7
2.2.2. Multi-processing & Distributed Computing	8
2.3. Parallel-in-Time Integration	9
3. Methods	10
3.1. Scaling the Parareal Algorithm	11
3.1.1. Preparing the subproblems	11
3.1.2. Solving the subproblems	13
3.1.3. Solving the root problem	16
3.1.4. Converging the root solution	18
3.2. Implementing the Parareal Algorithm at Scale	20
3.2.1. Massively Parallelizing the Parareal Algorithm	21
3.2.2. Distributing the Parareal Algorithm	23
4. Analysis	27
4.1. Numerical Analysis	27
4.1.1. Convergence	27
4.1.2. Stability	27
4.1.3. Error	27
4.2. Algorithm Analysis	27
4.2.1. Analysis of Parallel Algorithms	27
4.2.2. Time Complexity	27
4.2.3. Space Complexity	27
4.3. Benchmarks	27
5. Particle Production in Analog Cosmologies	28
6. Conclusion	29
6.1. Future work & Possible Optimizations	29
Bibliography	30

1. Introduction

2. Background

- 1) Equations of Motion
- 2) High-Performance Computing
- 3) Parallel-in-Time Integration

2.1. Equations of Motion

According to classical mechanics, the motion for any and every object in the universe can be determined for all time using only its current position, current velocity, and the forces acting on it [1] .

2.1.1. Initial Value Differential Equations

- Ordinary Differential Equations (ODE)
 - uncoupled
 - coupled
- Partial Differential Equations (PDE) e.g. wave equation

2.1.2. Traditional Numerical Methods

- ODEs
 - Predictor-Corrector Methods
 - The PA is not the first of its kind to follow this “predict-correct-loop” structure. In fact, there is a whole class of integration algorithms known as *predictor-corrector* methods.
 - Hartree-Fock Method (i.e. Self-Consistent Field Theory) is similarly iterative to the PA but iterations are done to minimize energy according to the variational principle of quantum mechanics.
 - Symplectic Integration
 - Traditional Methods in evolving Equations of Motion
 - Def don’t use Runge-Kutta methods
 - Symplectic Euler
 - Velocity Verlet
 - etc.
- PDEs
 - Method of Lines
 - Method of Relaxation

One of the most important algorithms used in evolving equations of motion is the velocity Verlet method.

2.2. High-Performance Computing

Some key aspects of high-performance computing (HPC) are:

- the difference between processes and threads
- the difference between a CPU “core” and GPU “core”

2.2.1. Multi-threading & GPU Computing

- CPU multithreading
- GPU multithreading & CUDA
- GPUs provide significant performance increase for parallel workloads over CPUs because there are more cores
- different brands have different GPGPU APIs such as NVIDIA with CUDA, AMD with ROCm, and Intel adhering to the oneAPI standard [2]
- can use high-level abstractions such as OpenMP [CN](#), OpenACC [CN](#), or language extensions such as Julia’s GPU ecosystem [CN](#) including CUDA.jl [CN](#)
- Context switching on a GPU is effectively free.

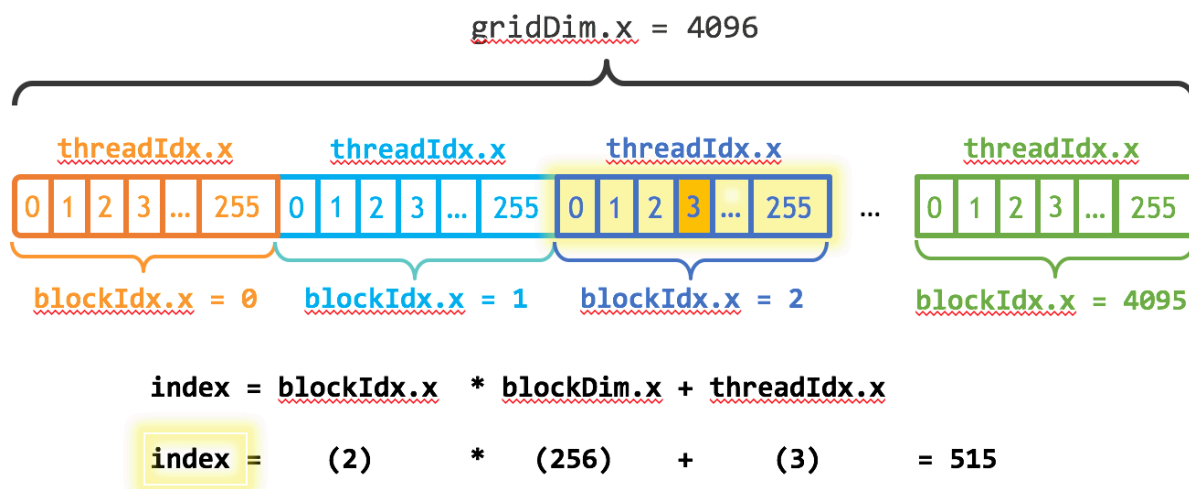


Figure 1: Each thread is assigned an index of the array (`index`) based on its location in its block (`threadIdx.x`), how many threads there are in its block (`blockDim.x`), and the block’s location in the grid (`blockIdx.x`). The cells in the image above represent cells of the array to which the labeled thread will write. Image credit [3].

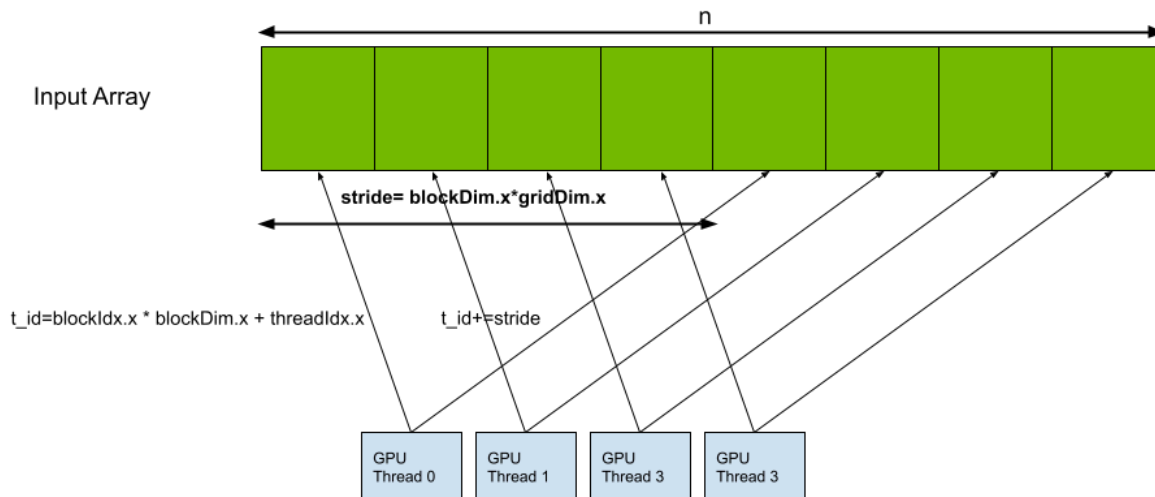


Figure 2: When there are more cells in the array than there are threads in the GPU, each thread processes multiple array cells. Once each thread is finished writing to its cell, it “jumps over” all the cells that were just written to by all the other threads in all the other blocks, and writes to the next one. The number of cells the thread “jumps”, i.e. the *stride*, is determined by the number of threads in each block (`blockDim.x`) and the number of blocks in each grid (`gridDim.x`). This is known as *index striding* and is frequently used in GPU programming to process arrays of arbitrary dimension [4]. Image credit [5]

2.2.2. Multi-processing & Distributed Computing

- Message Passing & Remote Call Procedure (RPC)
 - An RPC is effectively when one process tells another “Here’s an explicit list of instructions i.e. a recipe. Do it without thinking.”
- Because each process has its own memory space, each process must independently load any and all libraries, files, binaries, etc. it needs.
- Each host effectively needs to be an identical copy of the head node. This can be achieved by each host referring to a shared file system so they all manifestly the same binaries, versions of packages, etc. This needs to happen because an RPC can be thought of as sending a chunk of raw, textual source code to be run on the other process and/or machine. If that source code calls some functionality that is not loaded or otherwise available in that process, that call will error. Thus things like a GPU library must be not only available on each machine, but also loaded on each process.

2.3. Parallel-in-Time Integration

There are 3 traditional ways to parallelize the solution of a computational problem: CPU parallelization, GPU parallelization, and Distributed computing. While CPU parallelization is more straightforward to implement, GPU parallelization can allow for runtimes to decrease by many orders of magnitudes. Even lower run times can be achieved by combining either of these parallelization schemes with running them on multiple machines. This investigation focuses on parallelizing the solution of equations of motion using GPUs and multiple machines.

These approaches can offer massive increases in performance, but only for problems that are well-posed to be parallelized. Traditionally, initial value problems have been unable to be parallelized due their dependence on causality. Several methods have been created to overcome this limitation. These methods include the Parareal algorithm, Multigrid Reduction in Time (MGRIT), Parallel Full Approximation Scheme in Space and Time (PFASST). This investigation focuses on the Parareal algorithm.

Parareal

- The Parareal method has mostly been applied to first-order ordinary differential equations.
- Part of the novelty of this work is that it focuses on building support for second-order ODES

3. Methods

Simulating physical processes has traditionally been done sequentially; even during the modern age of hardware supporting parallel execution, using computers to calculate the evolution of physical phenomena has been sequential. Why haven't scientists just started doing things in parallel? Because of that pesky thing call *causality*; *the ball must go up before it can come down*. Because of this temporal dependence (spatial dependence has had its own workarounds such as the Barnes-Hut algorithm [6,7]), simulation of large-time-scale physics has thus taken a long time to execute. The Parareal algorithm (PA), and other parallel-in-time integration algorithms, have been developed in the last few decades to specifically address this issue [8]. Many details and variations of the Parareal algorithm have been investigated to find and address issues such as stability **CN**, convergence rates **CN**, application to higher-order differential equations **CN**. The main goal of this investigation is to contribute another variation: an implementation of the Parareal algorithm using methods from high-performance computing.

Before the PA can be implemented using these high-performance methods, the algorithm must be decomposed into its central components. The Parareal algorithm begins by partitioning a single IVP into several IVPs on smaller domains via an initial, inaccurate, “root” solution. Then each of the “subproblems” are solved using a sequential, accurate method on different threads at the same time. The final data for each of the subsolutions is then combined with the respective data of the root solution to yield a more accurate (i.e. “corrected”) root solution. This new root solution is then used to repeat the process until convergence.

The interpretation of the PA in terms of these recursive subproblems makes the algorithm *almost* embarrassingly parallel; the corrections to the root solution need to be done sequentially. In addition to this structure, the algorithms being evaluated in parallel manifestly depend on simple arithmetic; because of this simplicity, the PA is well-suited to be evaluated on the GPU. Likewise, distributed methods can be combined with GPU evaluation for further parallelization for either a single model (taking advantage of the recursive nature of the PA) or a system of models.

This chapter begins by casting the Parareal Algorithm in a form that is conducive to being scaled. The main idea is to recast the “magical” mechanism of the PA to something that can be executed recursively. In other words, the PA takes an IVP and produces a collection of IVPs, which can each be given to another instance of the PA. The latter half of this chapter is devoted to presenting a model for which the scalable version of the PA can be implemented. This model includes how the performance of the PA can be increased by using a GPU on a single machine, as well as how to build and use a cluster of machines to further increase performance.

Example: To help understand and clarify the mechanisms of the scalable PA, including the important details that are not explicitly covered in the algorithm itself, an example problem is used. Consider the motion of a thrown ball just after it leaves the hand over the course of 8 seconds (ignoring air resistance).

This motion is modeled by: $P = \left\{ \underbrace{\partial_t^2 \vec{r} = \vec{g}}_{\text{Acceleration}}, \underbrace{\vec{r}(0) = \vec{r}_0, \vec{v}(0) = \vec{v}_0}_{\text{Initial values}}, \underbrace{[0s, 8s]}_{\text{time span}} \right\}$. The machine has 8 cores.

3.1. Scaling the Parareal Algorithm

The Parareal Algorithm aimed to solve the problem of physics taking too long to simulate; it didn't.

The main idea of the Parareal algorithm (PA) is to break up a single IVP into many smaller IVPs using some low-accuracy solution, solve those in parallel using high-accuracy methods, correct your initial solution using the sub-solutions, then make a new low-accuracy solution based on the corrected data, and repeat this process until the solution doesn't change. The end result of this procedure is a solution identical to one produced by directly using the high-accuracy method while potentially taking a less time [9]. Because the PA wraps traditional (sequential) solvers, it could be considered a “meta-” or “higher-order” method to solve IVPs.

3.1.1. Preparing the subproblems

Let the second-order initial value problem P be defined such that

$$P = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [t_0, t_0 + \Delta t]\}, \quad (1)$$

and $D = [t_0, t_0 + \Delta t]$ is the closed time interval from t_0 to $t_0 + \Delta t$. The discretization N of P should be determined by the number of available threads N_t such that $N = mN_t$, for some positive integer m . The discretization should be chosen in this manner for maximum performance and efficiency; if $N = mN_t + r$, and $0 < r < N_t$, each thread will solve a subproblem m times until on the $m + 1$ iteration where only r threads would be active while $N_t - r$ threads idle (assuming all threads are synchronized). This type of optimization is sometimes referred to as “*flooding the threadpool*” to mitigate *thread starvation* **CN**.

With the discretization decided, partition the time domain D into subdomains D_p such that

$$D_p = \left[t_0 + \frac{p}{N}\Delta t, t_0 + \frac{p+1}{N}\Delta t \right] = [t_p, t_{p+1}]. \quad (2)$$

Use an fast integration method $\mathcal{G}_0$ (such as the Euler method) to compute initial root solutions $\{u_p^0\}_p$, $\{v_p^0\}_p$ defined such that

$$\{u_p^0\}_p = \{u_0^0, u_1^0, u_2^0, \dots, u_{N-1}^0\} \quad (3)$$

$$\{v_p^0\}_p = \{v_0^0, v_1^0, v_2^0, \dots, v_{N-1}^0\} \quad (4)$$

and $\{u_p^0, v_p^0\} = \mathcal{G}(\Delta t, u_{p-1}^0, v_{p-1}^0, \partial_t^2 u)$.

Subproblems P_p take the same from as in Eq. 1 (this also means subproblems are themselves, problems), but instead using initial conditions defined by those in the initial root solution. For example, the subproblem P_1 for the second ($p = 1$) subdomain, takes the form

$$P_1 = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_1^0, \partial_t u(0) = v_1^0, \quad [t_p, t_{p+1}]\}. \quad (5)$$

Alg 1 shows the pseudocode of this process for a given IVP and integration algorithm i.e. “propagator”, resulting in root solutions and the collected subproblems. With these subproblems in hand, the PA continues to its next stage: propagating these problems in parallel.

Alg 1: PREPARE THE SUBPROBLEMS

INPUT: Second order initial value problem P , Coarse solver C
OUTPUT: Solution for P made from `pos_seq` and `vel_seq`, and array of subproblems `subproblems`
// choose discretization
1 $N \leftarrow$ number of subproblems *// e.g. multiple of # of computer cores*
// partition the domain of P into N subdomains e.g. $[a, b] \rightarrow \{[a, c], [c, b]\}$
2 `subdomains` $\leftarrow$ `partition(P.domain)`
// use coarse solver to get positions and velocities for the root problem
3 `pos_seq, vel_seq` $\leftarrow$ `propagate(P, C)`
// create subproblems
4 **for** i from 1 to $N - 1$
5 `subdomain` $\leftarrow$ i -th domain partition `subdomains[i]`
6 `pos0` $\leftarrow$ initial position for i -th subproblem `pos_seq[i]`
7 `vel0` $\leftarrow$ initial velocity for i -th subproblem `vel_seq[i]`
8 `subproblems[i]` $\leftarrow$ ivp on `subdomain` with initial values `pos0` and `vel0` for acceleration `P.acc`
9 **return** Solution for root problem with `pos_seq` and `vel_seq`, and array of subproblems `subproblems`

Alg 1: Prepare the subproblems

Example: Given the example IVP (Eq. 5) and the available threads,

- 1) There should be 8 subproblems because there are 8 available threads.
- 2) Create the 8 time sub-domains $[0, 1], [1, 2], \dots, [7, 8]$.
- 3) Use the initial position and velocity to quickly solve the root problem via the Euler method to give a sequence of 8 total positions $\vec{r}_p^0$ and a sequence of 8 total velocities $\vec{v}_p^0$.
- 4) Use the calculated positions and velocities as initial positions and velocities to create 8 subproblems on the associated subdomains following the form

$$P_p = \{\partial_t^2 \vec{r} = \vec{g}, \quad \vec{r}(0) = \vec{r}_p^0, \quad \vec{v}(0) = \vec{v}_p^0, \quad [t_p, t_{p+1}]\}. \quad (6)$$

The result of this process is shown in Figure 3.

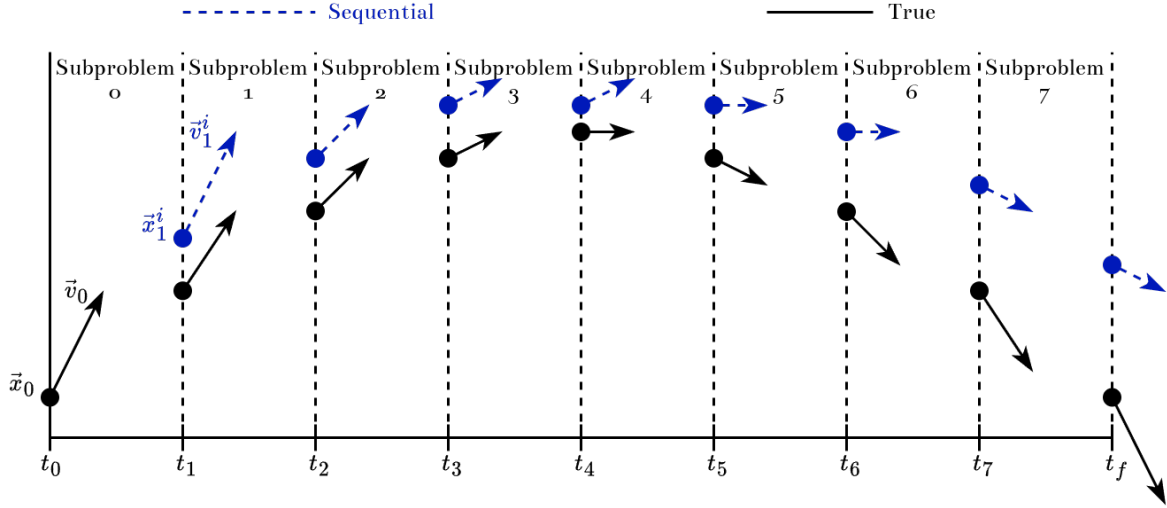


Figure 3: The motion of a ball flying through the air can be partitioned in time to form several initial value problems, each with its own initial position and velocity (upper, blue) determined by a fast integration method. Compared to the true solution (lower, black), this solution is very inaccurate.

3.1.2. Solving the subproblems

Discretizing the domain and propagating initial values as in the previous section constitutes the application of the *coarse propagator* $\mathcal{G}$ to the root problem. Because each of the produced subproblems is independent of the others, each can be accurately solved in parallel using a *fine propagator* $\mathcal{F}$ to reduce the total runtime by a factor equal to the number of subproblems. In other words, if applying $\mathcal{F}$ to a single subproblem has a runtime $\tau_{\mathcal{F}}$, then applying $\mathcal{F}$ to the root problem directly has a runtime $N * \tau_{\mathcal{F}}$ because there are N subproblems, whereas applying $\mathcal{F}$ in parallel only results in a runtime $\tau_{\mathcal{F}}$ because each application of $\mathcal{F}$ executes at the same time.

In general, a **propagator** $\mathcal{P}$ is defined by two key components: its integration algorithm $I_{\mathcal{P}}$, and its discretization $N_{\mathcal{P}}$. More specifically, the propagator $\mathcal{P}$ can be interpreted as a higher-order function mapping integration algorithms and discretizations to functions that, when applied to an IVP, reduce to sequences $\{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t$ of time-position and time-velocity pairs, respectively; the collection of the sequences is called the **solution** $S_{\mathcal{P}}$ of P . The solution can be interpreted as the set of function-graphs (as defined in [10]) of $\vec{r}$ and $\vec{v}$, and is defined such that

$$\mathcal{P}(I_{\mathcal{P}}, N_{\mathcal{P}})(P) = \left\{ \{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t \right\} =: S_P. \quad (7)$$

The application of the propagator to the subproblem is the core, or **kernel**, of the PA. While this description of the kernel is useful for understanding, it does not immediately lead to an algorithm that is well-suited for hardware-agnostic implementation (more details in section 3.2.1 on page 21). To that end, the implementation of the kernel presented here is composed of discretizing the subdomain and propagating the initial values separately.

Discretization: To avoid performance losses from each kernel allocating memory, each discretization kernel references a specific, pre-allocated, one-dimensional array δD of length $N_{\mathcal{P}}$. In order for the kernel to generate the appropriate samplings of the domain, δD has its first and last elements pre-populated with the values of the lower and upper bounds of that thread's assigned subproblem such that

$$\delta D = \{t_p, [N_{\mathcal{P}} - 2 \text{ arbitrary elements}], t_{p+1}\}. \quad (8)$$

With each kernel accessing this data, it can simply calculate and write the domain samples in-place; this process is shown in Alg 2.

Alg 2: PARALLEL DISCRETIZATION KERNEL

INPUT: Discretized domain **ddom** of length **N**

OUTPUT: Nothing

```

1 a, b ← (ddom[0], ddom[N-1])
2 step ← (b - a) / 2
3 for i from 1 to N - 2
4   ddom[i] ← a + (i - 1) * step
5 return
```

Alg 2: Each discretized subdomain is calculated by uniformly stepping from the lower bound to the upper bound. These results are written in-place.

Propagation: Much like the discretization kernel, the propagation kernel avoids memory allocations by using pre-allocated, pre-populated multidimensional arrays to store the position and velocity sequences; the initial values of the position and velocity for that kernel's subproblem are pre-populated as their respective arrays first element taking the form

$$\{\vec{r}_t\}_t = \{\vec{r}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\} \quad \{\vec{v}_t\}_t = \{\vec{v}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\} \quad (9)$$

Otherwise, the propagation kernel is no more than a traditional IVP solver as described in Section 2.1.2, but executed on many different subproblems simultaneously over many threads. This process is shown algorithmically in Alg 3.

Alg 3: PARALLEL PROPAGATION KERNEL

INPUT: A solver `solve`,
acceleration function `acc`,
sequence of `N` position vectors `pos_seq`,
sequence of `N` velocity vectors `vel_seq`
OUTPUT: Nothing

```

1 for i from 2 to N - 1
2   old_pos, old_vel  $\leftarrow$  pos_seq[i - 1], vel_seq[i - 1]
3   pos_seq[i], vel_seq[i]  $\leftarrow$  solve(old_pos, old_vel, acc, step)
4 return
```

Alg 3: Each subproblem is solved in parallel using traditional methods.

The Parareal Kernel: In summary, the parareal kernel (Alg 4) is launched on each thread simultaneously and uses the fine propagator to populate arrays prepared from the domain and initial values of that thread's problem. The domain is discretized by uniformly stepping from the lower bound of the problem's domain to the upper bound. The initial values are propagated using a traditional integration method (Figure 4). The result of this process is sequences of times, positions, and velocities that solve that thread's problem for different discretizations.

Alg 4: THE PARAREAL KERNEL

INPUT: a fine propagator `fine`, discretized domain `ddom`,
position sequence `pos_seq`, velocity sequence `vel_seq`
OUTPUT: Nothing

```

1 Use fine in the discretization kernel to populate to ddom
2 Use fine in the propagation kernel to populate pos_seq and vel_seq
3 return
```

Alg 4: Put it all together

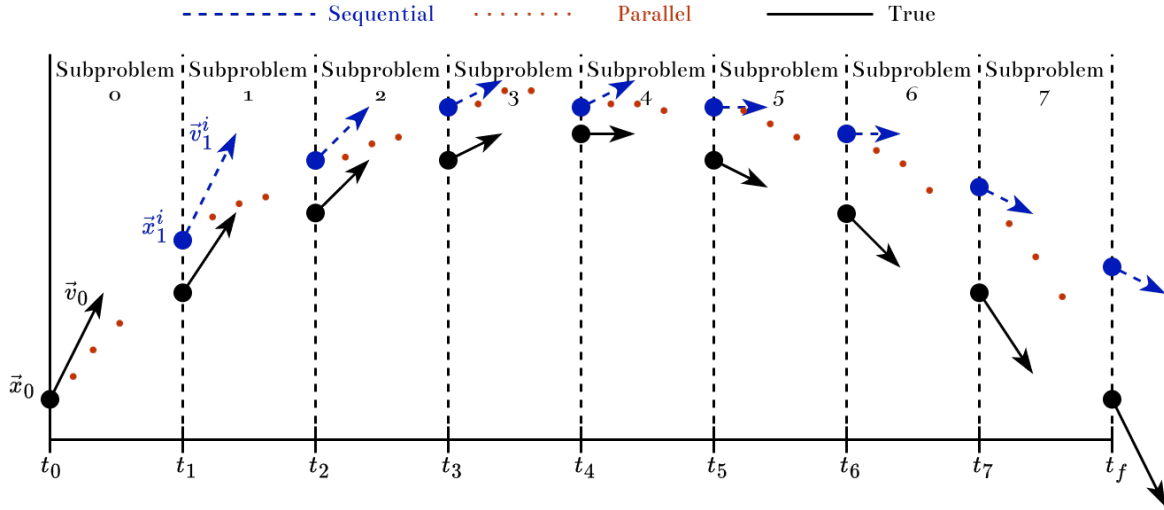


Figure 4: Each thread uses coarse and fine propagators to produce intermediate values (small, red dots) from the initial values (big, blue dots and arrows) of its assigned subproblem. Velocity data does exist, but is neglected here for visual clarity.

Example: For each of the 8 subproblems described by Eq. 6, each taking the form

$$P_p = \{ \partial_t^2 \bar{r} = \bar{g}, \quad \bar{r}(0) = \bar{r}_p^0, \quad \bar{v}(0) = \bar{v}_p^0, \quad [t_p, t_{p+1}] \}.$$

- 1) Define the fine propagator $\mathcal{F}$ as the Velocity-Verlet method with a discretization of $N_{\mathcal{F}} = 2^6 = 64$.
- 2) Prepare arrays
 - 2.1) Allocate $3 * 8 = 24$ arrays of length 64 for the fine domains, positions, and velocities.
 - 2.2) Populate the beginning and end of each domain array with the lower and upper bounds, respectively, of each problem's domain.
 - 2.3) Populate the beginning of each position array with the initial position of each problem.
 - 2.4) Populate the beginning of each velocity array with the initial velocity of each problem.
- 3) Launch the Parareal Kernel with the propagators and prepared arrays
 - **Note:** If there are more problems than threads, the kernel can index stride [4]; more on this in Section 3.2.1.

3.1.3. Solving the root problem

Because the root solution has been calculated via an inaccurate method, it can be made more accurate using the results of the fine propagator. Though the root solution is not corrected only with the results of the fine propagator, but rather by coarsely propagating the root initial values again but adding a corrector determined by a combination of the results of the fine propagator and the previous iteration's root solution. The main idea of this process is known as *Deferred Corrections* [11].

The correction phase, as defined in literature [8], takes the deceptively-simple recursive form

$$u_t^i := \underbrace{\mathcal{G}(u_{t-1}^i)}_{\text{predictor}} + \underbrace{\mathcal{F}(u_{t-1}^{i-1}) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{corrector}}, \quad (10)$$

where $u = \vec{r}, \vec{v}$ represents either position or velocity of the root problem. If the coarse propagation terms are collected as $\Delta_i \mathcal{G}_t^i := \mathcal{G}_t^i - \mathcal{G}_t^{i-1}$, Eq. 10 can be interpreted as *shooting method in time* [9]. It should also be noted that because the values returned by the coarse propagator are identical in successive iterations i.e. $i \rightarrow i+1 \Rightarrow \mathcal{G}(u_{t-1}^i) = \mathcal{G}(u_{t-1}^{i-1})$, they can be memoized and not calculated again (see Alg 5), leading to performance increases at the cost of storage space [12].

One way to interpret the correction equation is “at face value” as

On the current iteration, use the coarse propagator to recommend¹ what this value should be. Then correct it by adding the difference between the fine and coarse predictions from the previous iteration. Record this sum as the actual value.

An alternative interpretation arises from, effectively, “moving the correction to the top of the loop” as

Use the coarse propagator to traditionally evolve the root problem, but with an offset. This offset is initially zero.

This interpretation changes “use the propagator, then correct the value” to “correct the propagator, then use the value”.

Thus the overall behavior of Eq. 10 could be defined through the explicit recurrence relation

$$\begin{aligned} u_t^i &= \mathcal{G}_t^i(u_{t-1}^i) \\ u_0^i &= u(0). \end{aligned} \quad (11)$$

Here the coarse propagator is effectively parameterized and can vary throughout iterations and times such that $\mathcal{G}_t^i(u_{t-1}^i) := \mathcal{G}(u_{t-1}^i) + \Delta_t^i$ and

$$\begin{aligned} \Delta_t^i &= \mathcal{F}(u_t^i) - \mathcal{G}(u_t^i) \\ \Delta_t^0 &= 0. \end{aligned} \quad (12)$$

Eq. 11 has the same structure of a traditional propagation making it much simpler to understand its behavior. This understanding is made even easier when considering Eq. 12 as it allows the Eq. 11 to also define the original coarse propagation of the root problem. In other words, the corrected coarse propagator has a correction of zero on the original iteration. While this interpretation is easier to understand, the result is mechanically identical to Eq. 10.

¹This terminology is inspired by Giordano & Nakanishi’s interpretation of the Gauss-Seidel and Simultaneous Over Relaxation methods in their seminal text on computational physics [13].

Alg 5: NEW SOLUTION GENERATOR

INPUT: Coarse solver `coarse`,
 Coarse position array `cpos`, Coarse velocity array `cvel`,
 Fine position array `fpos`, Fine velocity array `fvel`,
 Root position array `psol[i-1:i, :]`, Root velocity array `vsol[i-1:i, :]`
OUTPUT: Nothing
// evaluated at iteration i
 1 `psol[i, 0], vsol[i, 0] ← p0, v0`
 2 **for** `t` from 1 to `N-1`
 3 `ppred[i, t], vpred[i, t] ← coarse(psol[i, t-1], vsol[i, t-1])`
 4 `psol[i, t] ← ppred[i, t] + fpos[i-1, t] - cpos[i-1, t]`
 5 `vsol[i, t] ← vpred[i, t] + fvel[i-1, t] - cvel[i-1, t]`

Alg 5: New position and velocity solutions are generated by coarse propagating the previous solution in the current iteration and combining it with the difference between the fine and coarse solutions from the previous iteration. The new solutions are pushed to the end of the solution arrays.

Example:

- 1) Have arrays of positions and velocities at each time for the previous and current iteration; the first element of the current iteration's array is the initial value of the problem, while the rest of empty. Also have arrays of positions and velocities generated from the coarse and fine propagators at each time for the previous iteration.
- 2) Use the new solution generator algorithm with these arrays and the coarse propagator.
- 3) The arrays for the position and velocity of the root solution at the current iteration are now populated.

3.1.4. Converging the root solution

Finally, as own in Alg 6, launch the parareal kernel (Alg 4) to gather the fine solutions for each point in time, and construct the new root solution (Alg 5) until the root solution stops changing between iterations. While there are many choices that can serve as valid convergence criteria [9], one of the simplest is:

$$\max_{1 \leq t \leq N-1} |u_t^i - u_t^{i-1}| < \varepsilon, \quad (13)$$

for some threshold ε . Eq. 13 determines convergence when every point in the solution changes by less than some amount between iterations.

Alg 6: THE PARAREAL ALGORITHM

INPUT: Root problem P , Coarse propagator G , Fine propagator F , Convergence threshold ϵ_p
OUTPUT: Discretized domain ddom , Root position sequence pos , Root velocity sequence vel

```

1  $\text{ddom}, \text{pos}[0, :], \text{vel}[0, :] \leftarrow$  Prepare the subproblems via a coarse solution
2  $i \leftarrow 1$ 
3 while  $\max(\text{changes}) \geq \epsilon_p$ 
4    $\text{fpos}[t], \text{fvel}[t] \leftarrow$  Launch the parareal kernel in parallel with  $F$ ,  $\text{pos}[i-1, :], \text{vel}[i-1, :]$  to get
   the fine solutions for subproblem  $t$ 
5    $\text{pos}[i, :], \text{vel}[i, :] \leftarrow$  Construct new root solutions with  $G$ ,  $\text{pos}[i-1, :], \text{vel}[i-1, :]$ , and
    $\text{fpos}, \text{fvel}$ 
6    $\text{changes} \leftarrow$  The difference between the current and previous solutions
7 return  $\text{ddom}, \text{pos}, \text{vel}$ 
```

Alg 6: The Parareal Algorithm is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

The magic of the Parareal algorithm lies in its divide-and-conquer approach to solving initial value problems. The “root” problem is sequentially and inaccurately solved to divide it into smaller problems whose initial values are defined by the solution. Those problems are simultaneously and accurately solved in parallel. The root problem is then solved in the same way as before, but at each step, the data is modified by combining the previous accurate and inaccurate solutions. Finally, the new root solution defines new problems, and the loop continues until the the solution has converged.

3.2. Implementing the Parareal Algorithm at Scale

Surely more threads is the answer?

Section 3.1 highlights the PA’s nature of being quasi-embarrassingly-parallel i.e. the performance of the algorithm scales with the number of threads, while still being bottlenecked by a periodic sequential process. That being said, the previous discussion ignores the details and nuances of implementing the PA including the actual form of the threads. The primary goal of this work is to provide two new implementation models that both take advantage of increased parallelism and allow easy scaling: using the massively parallel architecture of graphics processing units (GPUs), and the scalability of distributed systems. Instances of these implementations are also provided [14].

The PA as defined in Alg 6 does not change in it *what* it does when implemented to use GPUs, but rather *how* it does. There are several issues that arise when utilizing general-purpose GPU computing (GPGPU) such as the GPU needing to wait until the CPU tells it to do something, better performance with less precision, and the restriction to using primitive types like “ints” and “floats”. Though, the biggest issue is the need to consider the movement of data between RAM and VRAM, or more generally host memory and device memory; considering unshared memory spaces will be even more important in section Section 3.2.2.

The distributed-based implementation focuses on distributing problems across multiple remote machines. These machines solve their problems simultaneously with the other machines, thus achieving a form of parallelism only limited by the number of accessible machines. These problems could either arise from the coarse propagation of a single root problem, yielding a “problem tree” (Figure 5) where each machine would create-distribute-collect its own set of problems, or if there are multiple “true root” problems e.g. a system of ODEs.

The GPU- and distribution-based methods can be combined to further parallelize solving an initial value problem. If each of the machines available to the distributed network has at least one GPU (a single machine can have multiple; more details in Section 3.2.2), this implementation will automatically identify, manage, and use all of them. Thus these methods can be composed to provide a scalable model of parallel-in-time integration for equations of motion.

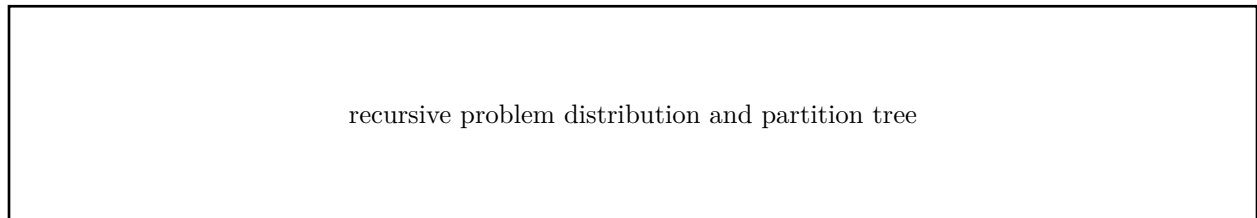


Figure 5: The problems can be distributed providing a recursively parallelized solution.

3.2.1. Massively Parallelizing the Parareal Algorithm

The GPU-based implementation focuses on three ideas. The first is utilizing the massively-parallel architecture of a GPU to *simultaneously* use orders-of-magnitude more threads than what would be possible with a CPU. The second is considering the movement of data between the host memory (RAM) and the device memory (VRAM). And the last is needing to use primitive data-types. Otherwise, the underlying algorithm is no different than what is presented in Section 3.1.

The PA (Alg 6) is only limited by the number of threads at its disposal. When the number of threads is greater than the number of cores, the processor needs to switch thread contexts in order to balance the evolution of each thread. On a CPU, this context switching is very costly and can lead to drastic decreases in performance [15,16]. On a GPU however, switching thread-contexts is nearly free [17], allowing there to be *many* more threads than processors without sacrificing efficiency. So, it is very beneficial to execute the PA on hardware that not only can efficiently handle many threads, but also have them running at the same time.

All relevant data is first allocated and pre-populated by the CPU on the host. Then the CPU copies that data to the device. Then the CPU tells the GPU to execute the parareal kernel on its copy of the data, producing solution data. The CPU then copies the solution data from the device to the host and recreates the problems. The transfer of data (and the CPU launching the kernel on the GPU) between the host and the device serves as the main performance bottleneck in this process. [17] Dynamic parallelism can be used to launch kernels directly from the GPU, thus circumventing the performance drawbacks of host-device communication [17].

GPUs can only process primitive types of data such as integers, floats, booleans, and other “bits-types”. This precludes collecting the problem and solution data in more intuitive forms like one would do when representing them mathematically. In other words, whereas a CPU is happy to handle several boxes each with its own set of elements e.g. domain, acceleration, initial position, and initial velocity, GPUs need this same underlying data to be collected such that all domains are in one box, all acceleration functions are in another box, all initial positions are another, and initial velocities in another. These “boxes” take the form of arrays. It is for this reason, the PA as described in Section 3.1 uses its data as arrays.

So, why is the PA well-suited to be implemented to use GPUs? Because the data is only composed of numbers, it can be simply represented in a GPU-friendly array structure. The massive number of cores on a GPU can simultaneously process these arrays with a much lower cost of switching between threads and problems. And finally, the solution data can be easily copied back to the host. This process is shown in Alg 7 and Figure 6.

Alg 7: THE GPU-BASED PARAREAL ALGORITHM

INPUT: Root problem P , Coarse propagator G , Fine propagator F , Convergence threshold ϵ_p
OUTPUT: Discretized domain ddom , Root position sequence pos , Root velocity sequence vel

```

1  $i \leftarrow 0$ 
2  $\text{ddom}, \text{pos}[i, :], \text{vel}[i, :] \leftarrow$  On the host, prepare the subproblems via a coarse solution
3 while  $\max(\text{changes}) \geq \epsilon_p$ 
4    $i += 1$ 
4   // fine propagate on the device
5    $\text{posd}[i-1, :], \text{veld}[i-1, :] \leftarrow$  Copy  $F$ ,  $\text{pos}[i-1, :], \text{vel}[i-1, :]$  to the device
6    $\text{fposd}[t], \text{fveld}[t] \leftarrow$  Launch the parareal kernel to get the fine solutions for subproblem  $t$ 
7    $\text{fpos}, \text{fvel} \leftarrow$  Copy the fine solutions  $\text{fposd}, \text{fveld}$  to the host
7   // coarse propagate on the host
8    $\text{pos}[i, :], \text{vel}[i, :] \leftarrow$  Construct new root solutions with  $G$ ,  $\text{pos}[i-1, :], \text{vel}[i-1, :], \text{fpos}, \text{fvel}$ 
9    $\text{changes} \leftarrow$  The difference between the current and previous solutions
10 return  $\text{ddom}, \text{pos}, \text{vel}$ 

```

Alg 7: The Parareal Algorithm is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

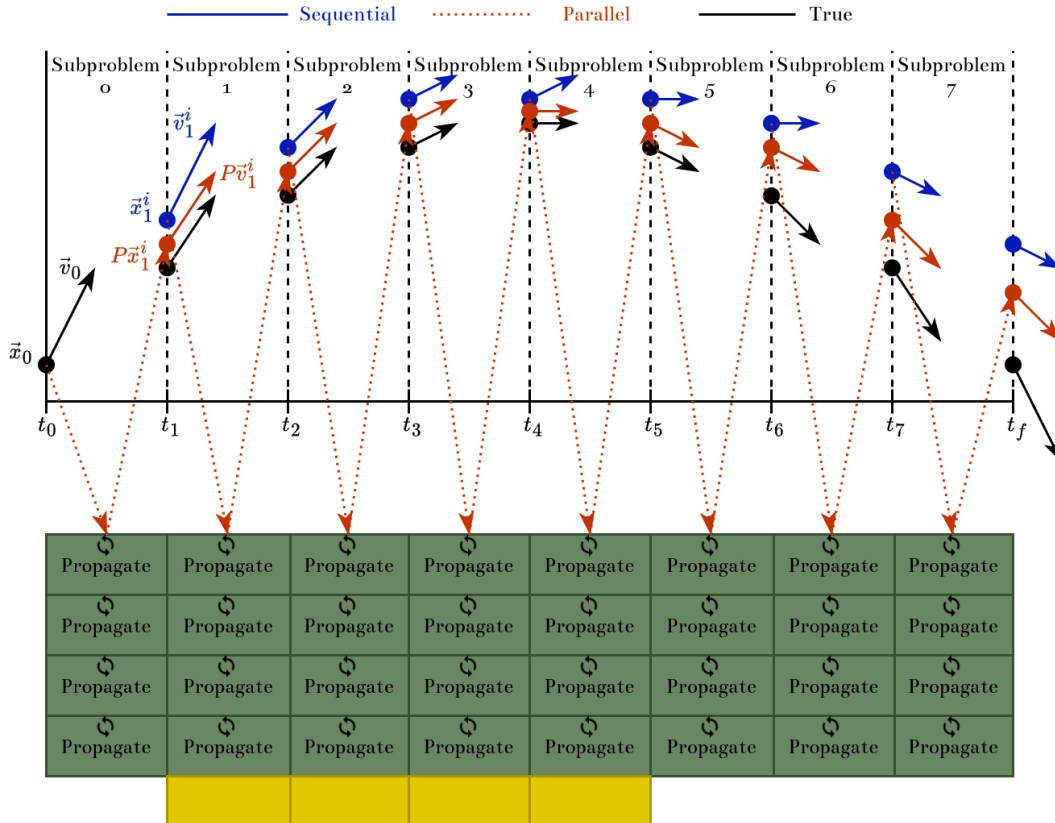


Figure 6: Sequential solutions (top in blue) are sent to the GPU to be finely-propagated (mid in red) in parallel; true solutions (bottom in black) are shown for comparison.

3.2.2. Distributing the Parareal Algorithm

While the PA can be further parallelized using GPUs, the fact still stands that the PA is quasi-embarrassingly-parallel. In other words, each subproblem is independent of the others while each is being solved, and each of these subproblems can be assigned its own thread. So, if there are more threads available, higher performance or accuracy can be achieved. The implementation presented here provides more threads by sending problems to remote machines where they can be run simultaneously; in other words, multiple machines with their own CPUs and GPUs are networked together to form a *cluster* where the work is distributed amongst all machines.

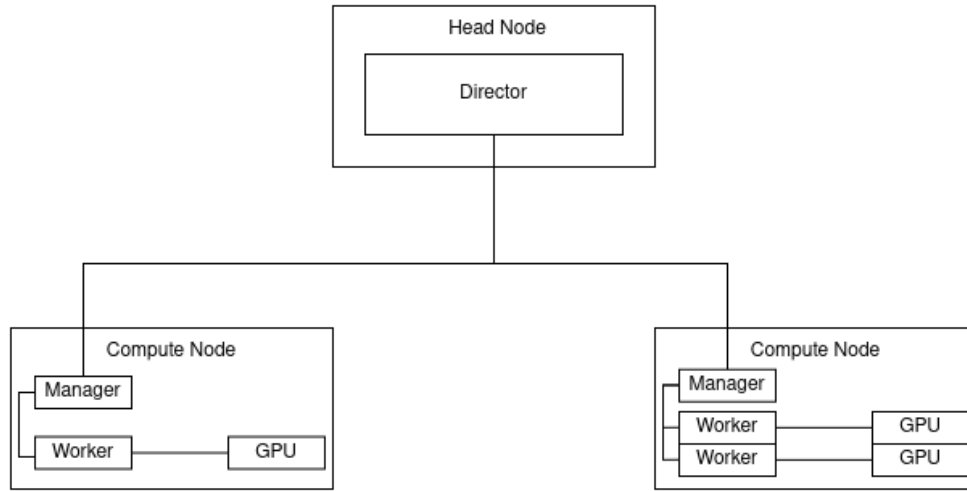


Figure 7: The assumed topology of the cluster presented in this work.

While clusters can take many forms **CN**, this implementation considers building a cluster from the ground up in a modular and ad-hoc manner. This way the cluster can theoretically scale without limit. The general structure, or *topology*, of the cluster is rather simple: A *head node* runs the *director* process, which tells *worker* processes on other *compute nodes* what to do; this structure is shown in Figure 7. In other words, the director only decides and does not do, while the workers do not decide and only do. While the workers can solve their problems and communicate with every other worker (and the director), only the director can spawn new processes. Once the director has finished spawning and configuring the workers as in Alg 8, the director moves on to begin the PA.

Alg 8: PREPARE THE CLUSTER

INPUT: A collection of hosts not ready to compute

OUTPUT: A collection of hosts ready to compute

- 1 Director process spawns manager processes on each host
 - 2 Each manager sends the number of devices on its host back to the director
 - 3 Director process spawns worker processes on each host, 1 for each device on that host
 - 4 Pair each worker with a device on its host
-

Alg 8: The cluster can be created and prepared in 4 simple steps.

In order for this implementation to be flexible, the director does not assume any a-priori configuration of any worker nodes. This way any node can be seamlessly introduced to the cluster. The drawback of this flexibility is that the cluster must be created each time. Part of this creation is identifying the available devices in the cluster. To do this, the director first uses a user-given collection of hostnames to spawn a single “manager” process on each of the given hosts.

Alg 9: SPAWN MANAGER PROCESSES

INPUT: A list of hosts

OUTPUT: A list of manager process IDs

- 1 The director spawns manager process on each host
 - 2 The director tells each manager to load the Parareal library
-

Alg 9: Manager processes are created to identify the number of devices on a host, and can manage the network communication between hosts.

Once the managers have been spawned, the director asks them how many devices are on their host. The manager measures this and sends back the information to the director. The director, because it is the only process that can spawn workers, spawns a number of workers on each host equal to the number of devices on it. This process is shown in Alg 10.

Alg 10: SPAWN WORKER PROCESSES

INPUT: A list of manager IDs

OUTPUT: A list of worker IDs

- 1 The director tells each manager to load the GPU library // *provides ability to count devices*
 - 2 The director requests the number of devices on the host from each manager
 - 3 For each host
 - 4 | For each device on the host
 - 5 | | The director spawns a process on the host
 - 6 The director tells each manager to load the Parareal library
-

Alg 10: Worker processes are spawned by the director on each host based on the number of devices available to that host.

Once the workers are spawned on their respective hosts, each of them needs a device. While the fundamental idea of assigning a device to a worker is trivial (as shown in Alg 11), the implementation suffers from the fact the a device should not be assigned to a process on a different host! In this implementation, process IDs correlate to the order in which they were spawned e.g. process 2 was spawned second, process 3 was spawned third; in other words, the process IDs are relative to the whole cluster. The device IDs, however, are relative to their host machine. So, care must be taken in order to pair processes and devices on the same host.

Alg 11: ASSIGN DEVICES - HIGH LEVEL

INPUT:**OUTPUT:**

```

1 The director tells each new worker to load the GPU library // provides ability to assign devices
2 For each host
3   | For each worker on that host
4   |   | The worker assigns an on-host device to itself

```

Alg 11: Pairing workers and devices is trivial at a high level

For example

- The director has process ID (pid) 1 and is on its own host.
- Host X has pid 2, and host Y has pids 3, and 4.
- Host X has 1 device with ID 0, and host Y has 2 devices with ID 0 and 1.
- The desired result is
 - Device 0 on host X is assigned to pid 2
 - Device 0 on host Y is assigned to pid 3
 - Device 1 on host Y is assigned to pid 4

One way to address this issue is to create “host objects” by collecting the hostnames, pids, and number of devices for each host. The collection of these host objects, together with their relative connections, would constitute a “cluster object” or “cluster graph”. While the actual ids of the devices on each host are what is important, the pattern is the same for all hosts e.g. `devids = 0, 1, ..., ndevs-1`. This way only a single integer needs to be stored instead of a list.

Alg 12: CREATE HOST OBJECTS

INPUT: List of hostnames, list of managers, list of device counts**OUTPUT:** List of host objects `hosts`

```

1 For each host
2   | name ← name of host
3   | workers ← list of woker IDs on host
4   | ndevs ← number of devices on host
5   | hosts[i] ← Host(name, workers, ndevs)
6 return hosts

```

Alg 12: To aid in the pairing of devices and workers, host objects can be created to make sure devices are assigned to workers on the same host.

A host object packages together the worker IDs and number of devices on the same host. Thus devices can be assigned to workers on the same machine simply by iterating through the host objects. This process is shown in Alg 13.

Alg 13: ASSIGN DEVICES

INPUT: List of host objects `hosts`**OUTPUT:** Nothing

```

1 Load the GPU library on each worker // provides ability to assign devices
2 for host in hosts
3   workers  $\leftarrow$  host.workers
4   devids  $\leftarrow$  (0, 1, ..., host.ndevs-1)
5   for each pair (worker, devid)
6   |   Assign devid to worker

```

Alg 13: A more detailed algorithm of assigning devices to workers on the same host.

Once the devices are assigned, the cluster has been prepared. The director then becomes the driver of the PA, as shown in Alg 14. The director begins the PA as it normally would, but instead of either dispatching the parallel propagation to other CPU threads or the GPU, the director gives each worker a problem to propagate in parallel, either on its CPU or GPU. Since this dispatching takes little time, the director then waits until it gets the solutions from the the workers. Finally, the director coarse propagates the root problem with the solutions, checks if the solution has converged, and loops if it hasn't.

Alg 14: THE PARAREAL ALGORITHM AT SCALE

INPUT: A root problem, coarse and fine propagators, and a convergence threshold**OUTPUT:** A solution to the root problem*// on a prepared cluster*

```

1 The director coarse propagates root problem to get initial solution
2 While the root solution has not converged
3   The director creates problems
4   For each problem
5     The director sends the problem to a worker
6     The director tells the worker to solve the problem using the Parareal algorithm on its GPU
7   The director requests the solution from the worker
8   The director waits to get all solutions
9 The director coarse propagates root problem with corrections to get new solution

```

Alg 14: The Parareal Algorithm can distribute its subproblems amongs multiple machines
in order to achieve higher performance.

4. Analysis

4.1. Numerical Analysis

- use the pendulum i.e. the simple harmonic oscillator to test numerical analysis properties since we know the analytic solutions and can compare.

4.1.1. Convergence

4.1.2. Stability

4.1.3. Error

4.2. Algorithm Analysis

4.2.1. Analysis of Parallel Algorithms

4.2.2. Time Complexity

- A detailed analysis of the convergence rates of the PA has been done [9].
- Network communication is slow, and thus there's overhead to distributed computing.
 - Problems need to take long enough to make the overhead of network communication worth it.

4.2.3. Space Complexity

4.3. Benchmarks

5. Particle Production in Analog Cosmologies

- Solve the partial differential equation
- spectral decomposition
- system of equations $\partial_t^2 \tilde{\theta} - \left(\frac{\dot{a}}{a}\right) \partial_t \tilde{\theta} - ac^2 k^2 \tilde{\theta} = 0$ for wavenumber $k \leq k_c$

6. Conclusion

- Equations of motion can now benefit from parallel solvers.
- Certain problems are well-suited to a divide-and-conquer approach.
- Problems with “doubly parallel” characteristics can leverage both local and distributed parallelism, achieving significant computational efficiency.
- These advancements pave the way for modeling acoustics in expanding volumes.

6.1. Future work & Possible Optimizations

- Krylov enhanced subspaces
- CUDA dynamic parallelism
- Implement with C, Fortran, CUDA, NVSHMEM, MPI
- Make gpu-backend-agnostic with KernelAbstractions.jl
- this physics could be better done with PFASST
- non-dimensionalize everything following [18]
- use a variable size time discretization algorithm, then base integration of those differences
- Use dynamic parallelism to avoid the cpu having to launch the kernels
- Use dynamic parallelism to even perform the coarse propagation

Bibliography

- [1] L. D. Landau and E. M. Lifshitz, *Mechanics, Third Edition: Volume 1 (Course of Theoretical Physics)*, 3rd ed. (Butterworth-Heinemann, 1976).
- [2] A. Fortenberry and S. Tomov, *Extending MAGMA Portability with Oneapi*, in *2022 Workshop on Accelerator Programming Using Directives (WACCPD)*, Vol. 0 (2022), pp. 22–31.
- [3] M. Harris, *An Even Easier Introduction to CUDA*, (2017).
- [4] M. Harris, *CUDA Pro Tip: Write Flexible Kernels with Grid-Stride Loops*, (2013).
- [5] D. Singal, *N Ways to SAXPY: Demonstrating the Breadth of GPU Programming Options*, (2021).
- [6] J. Barnes and P. Hut, A hierarchical $O(N \log N)$ force-calculation algorithm, *Nature* **324**, 446 (1986).
- [7] T. Hamada, K. Nitadori, K. Benkrid, Y. Ohno, G. Morimoto, T. Masada, Y. Shibata, K. Oguri, and M. Taiji, A novel multiple-walk parallel algorithm for the Barnes–Hut treecode on GPUs – towards cost effective, high performance N-body simulation, *Computer Science - Research and Development* **24**, 21 (2009).
- [8] J.-L. Lions, Y. Maday, and G. Turinici, Résolution d'EDP par un schéma en temps «pararéel», *Comptes Rendus De L'académie Des Sciences - Series I - Mathematics* **332**, 661 (2001).
- [9] M. J. Gander and S. Vandewalle, Analysis of the Parareal Time-Parallel Time-Integration Method, *SIAM Journal on Scientific Computing* **29**, 556 (2007).
- [10] C. Pinter, *A Book of Set Theory* (Dover Publications, 2014).
- [11] B. W. Ong and R. J. Spiteri, Deferred Correction Methods for Ordinary Differential Equations, *Journal of Scientific Computing* **83**, 60 (2020).
- [12] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms, Fourth Edition* (MIT Press, 2022).
- [13] N. Giordano and H. Nakanishi, *Computational Physics* (Pearson/Prentice Hall, 2006).
- [14] N. Chapman, *PararealGPU.jl*, (n.d.).
- [15] W. Stallings, *Operating Systems: Internals and Design Principles* (Pearson Education, 2011).
- [16] C. Li, C. Ding, and K. Shen, *Quantifying the Cost of Context Switch*, in *Proceedings of the 2007 Workshop on Experimental Computer Science* (Association for Computing Machinery, San Diego, California, 2007), p. 2–es.
- [17] S. Cook, *CUDA Programming: A Developer's Guide to Parallel Computing with Gpus*, 1st ed. (Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2012).
- [18] H. Langtangen and G. Pedersen, *Scaling of Differential Equations* (Springer International Publishing, 2016).